

Détection efficace de propriétés statistiques de chiffrements symétriques

TER - Master 1 SeCReTS

Alexandre FRANÇOIS Binet DUPRÉ Bastien GUIBERT

figures/Logotype UPSaclay_RVB.png

- 1 Titre de la section 1
 - Algorithme Naïf
 - Algorithme Standard
- 2 Titre de la section 2
- 3 Introduction
- 4 Méthodologie
- 5 Résultats
- 6 Conclusion

Introduction à la cryptanalyse différentielle

- Introduite par Eli Biham et Adi Shamir à la fin des années 1980, connue d'IBM depuis les années 1970
- Repose sur l'observation de la propagation d'une différence fixée entre deux messages à travers une fonction de chiffrement.

Différence entre deux messages chiffrés

Soit x et x' deux messages clairs, on cherche une différence β entre leurs chiffréments, tel que :

$$F(x) \oplus F(x') = \beta$$

En exprimant x' en fonction de x et d'une différence α on trouve :

$$F(x) \oplus F(x \oplus \alpha) = \beta$$

On note ainsi la paire (α, β) qu'on appelle communément "différentielle"

Introduction à la cryptanalyse différentielle

Pour toute analyse de fonction par cryptographie différentielle on retrouve toujours au moins deux occurrences d'une paire (α, β) .

Preuve :

$$\begin{aligned} F(x \oplus \alpha) \oplus F((x \oplus \alpha) \oplus \alpha) &= F(x \oplus \alpha) \oplus F(x \oplus \alpha \oplus \alpha) \\ &= F(x \oplus \alpha) \oplus F(x) \\ &= \beta \end{aligned}$$

Une fonction où chaque paire apparaît zéro ou deux fois est une fonction APN (*Almost Perfectly Nonlinear*)

- S-box d'AES (quand n impair)

L'Algorithme Naïf Absolu

- Méthode la plus directe : évaluer de manière exhaustive toutes les paires possibles de textes clairs
- Construction de la **Table de Distribution des Différences (DDT)** complète

Principe

Pour chaque $\alpha \in [1, 2^n - 1]$ et chaque $x \in [0, 2^n - 1]$:

$$\beta = F(x) \oplus F(x \oplus \alpha)$$

On incrémente la case (α, β) de la DDT. $\alpha = 0$ est exclu (trivial)

$$\underbrace{\begin{array}{c} \alpha_0 \\ \alpha_1 \\ \vdots \\ \alpha_{n-1} \end{array} \begin{pmatrix} \begin{array}{c} x_0 \\ F(x_0) \oplus F(x_0 \oplus \alpha_0) \\ F(x_0) \oplus F(x_0 \oplus \alpha_1) \\ \vdots \\ F(x_0) \oplus F(x_0 \oplus \alpha_{n-1}) \end{array} & \begin{array}{c} x_1 \\ F(x_1) \oplus F(x_1 \oplus \alpha_0) \\ F(x_1) \oplus F(x_1 \oplus \alpha_1) \\ \vdots \\ F(x_1) \oplus F(x_1 \oplus \alpha_{n-1}) \end{array} & \cdots & \begin{array}{c} x_{n-1} \\ F(x_{n-1}) \oplus F(x_{n-1} \oplus \alpha_0) \\ F(x_{n-1}) \oplus F(x_{n-1} \oplus \alpha_1) \\ \vdots \\ F(x_{n-1}) \oplus F(x_{n-1} \oplus \alpha_{n-1}) \end{array} \end{pmatrix}}_{\text{DDT}}$$

L'Algorithme Naïf Absolu

Complexité de l'algorithme brut

- **Complexité en temps** : $\mathcal{O}(2^{2n})$
- **Complexité en données** : $\mathcal{O}(2^n)$
- **Complexité en mémoire** : $\mathcal{O}(2^{2n})$

Problème : Inutilisable en temps raisonnable dès que $n > 16$

Trois optimisations majeures (C++) :

- **Pré-calcul des chiffrés** : Stocker $F(x)$ dans une table T , puis $\beta = T[x] \oplus T[x \oplus \alpha]$ (deux accès mémoire au lieu de deux chiffréments)
- **Filtrage à la volée (pDDT)** : Ne sauvegarder que les différentielles dépassant un seuil de probabilité p , évitant le stockage en $\mathcal{O}(2^{2n})$
- **Parallélisation (OpenMP)** : Distribution du parcours des α sur les différents cœurs du processeur

Le « mur de la RAM » sur Speck 32/64 ($n = 32$) :

Saturation mémoire

La table de pré-calcul nécessite théoriquement 34,36 Go de RAM
⇒ Dépassement immédiat des 16 Go disponibles (*Out-Of-Memory*)

Stratégies d'adaptation :

- Suppression du pré-calcul ⇒ chiffrement à la volée (lent)
- Désactivation de la parallélisation ⇒ moins d'allocations simultanées
- Nettoyage mémoire à la volée ⇒ affichage immédiat + purge

- Transformation de la recherche déterministe en **échantillonnage statistique**
- Pour chaque α , on teste k paires aléatoires au lieu de 2^n
- Complexité en temps $\Rightarrow 2^n \cdot p^{-1}$

Modélisation statistique

Le nombre de succès X suit une loi de Poisson $\mathcal{P}(\lambda)$ avec $\lambda = k \times p$.

Les auteurs fixent $k = 4/p \Rightarrow \lambda = 4$

$$P(X \geq 2) = 1 - e^{-\lambda}(1 + \lambda) \approx 0,9085$$

$\Rightarrow$ Seuil de confiance de $\sim 90\%$

Critère de détection : $count \geq 2$

Signal (vraie différentielle, probabilité p)

$$\lambda = k \times p = 4 \Rightarrow P(X \geq 2) \approx 90\%$$

Bruit (permutation aléatoire, $p_{rand} = \frac{1}{2^n - 1}$)

Pour Speck 64 ($n = 64$, $p = 2^{-13}$): $\lambda_{rand} \approx 2^{-49}$

Par développement de Taylor : $P(X \geq 2) \approx \frac{\lambda^2}{2} \approx 2^{-99}$

⇒ Collision accidentelle statistiquement impossible

Complexité

Reste limité par le facteur 2^n : il faut itérer sur toutes les différences α possibles.

- Pour Speck 64/128 : 2^{64} itérations $\Rightarrow$ inabordable par un ordi personnel
- Nécessité de **briser la barrière du 2^n**

Solution : exploiter le **paradoxe des anniversaires** pour convertir la recherche en une recherche de **collisions** $\Rightarrow$ complexité en $2^{n/2} \cdot p^{-1}$

Titre de la section 2

1. Objectifs de l'étude

- 1 Modéliser le coût d'une cryptanalyse différentielle exhaustive
- 2 Étudier l'effet des S-Box sur un Feistel
- 3 Rechercher des clés faibles

Question centrale

Une S-Box performante localement améliore-t-elle forcément le chiffrement global ?

2. Structure du Feistel utilisé

Bloc de 12 bits séparé en deux moitiés de 6 bits.

À chaque round :

$$L_{i+1} = R_i$$

$$R_{i+1} = L_i \oplus F(R_i, K_i)$$

avec :

$$F(R_i, K_i) = SBox(R_i \oplus K_i)$$

- K_i : sous-clé dérivée de K
- S-Box bijective $6 \rightarrow 6$

3. Optimisation par Hill Climbing

Principe :

- 1 Génération d'une S-Box aléatoire
- 2 Échange de deux valeurs
- 3 Évaluation du score
- 4 Conservation si amélioration
- 5 Répétition sur plusieurs milliers d'itérations

4. Temps d'exécution observé

Nombre de couples testés :

$$2^n \times 2^n = 2^{2n}$$

Donc :

$$O(2^{2n})$$

5. Comparaison de deux S-Box

Critère	Base	Optimisée
max(DP)	0.2500	0.0938
min NL	15	24
Avalanche	0.743	0.969

Observation

La S-Box optimisée domine sur tous les critères locaux.

6. Impact global dans Feistel

Version	Best DP	Avg Top-5
Classique	0.003418	0.001271
Modifiée	0.003906	0.003711

Résultat important

Une meilleure S-Box locale n'améliore pas forcément le chiffrement complet.

7. Optimisation guidée par le chiffrement

Objectif : optimiser selon les performances du Feistel.

Méthode	Best DP	Avg Top-5
Initiale	0.003418	0.001271
In-cipher	0.001413	0.000842

Conclusion

L'optimisation globale produit une amélioration réelle.

8. Conclusion

- 1 La complexité suit bien une loi exponentielle
- 2 Une bonne S-Box locale ne suffit pas
- 3 Optimiser le système complet est plus efficace
- 4 Aucune clé faible détectée sur 4096 clés

Message final

Un composant cryptographique doit être évalué dans son architecture globale.